Documentación Técnica: Tarea 4 - Análisis Sintáctico Descendente (ASD)

Desarrollador: Juan Camilo Lozano Cortés

Lenguaje: Python

Módulo: Procesadores de Lenguajes

1. Preparación del Entorno:

Para el desarrollo de esta herramienta, se optó por una arquitectura modular en Python 3. A diferencia de la tarea anterior, no se utilizaron librerías externas (como Lark o Graphviz), ya que el objetivo era implementar desde cero la lógica matemática de los conjuntos de predicción mediante algoritmos recursivos y de punto fijo (iterativos).

2. Implementación de Algoritmos Core:

Se diseñaron tres componentes algorítmicos fundamentales para el análisis de gramáticas:

JuanLozanoGit Create main.py

Code Blame 31 lines (25 loc) · 1.14 KB

```python
from gramatica import EJERCICIO_1, EJERCICIO_2
from algoritmos import calcular_first, calcular_follow, calcular_predict

def mostrar_resultados(titulo, g, inicio):
    print(f"\n{'#'*30}\n# {titulo}\n{'#'*30}")

    # 1. Calcular FIRST
    primeros = {nt: calcular_first(nt, g, {}) for nt in g}
    print("\n[+] CONJUNTOS FIRST:")
    for nt, v in primeros.items(): print(f"    FIRST({nt}) = {v}")

    # 2. Calcular FOLLOW
    siguientes = calcular_follow(g, primeros, inicio)
    print("\n[+] CONJUNTOS FOLLOW:")
    for nt, v in siguientes.items(): print(f"    FOLLOW({nt}) = {v}")

    # 3. Calcular PREDICT
    predicciones = calcular_predict(g, primeros, siguientes)
    print("\n[+] CONJUNTOS DE PREDICCION:")
    for nt, p, res in predicciones:
        print(f"    P({nt} -> {' '.join(p)}) = {res}")

if __name__ == "__main__":
    # Ejecutar para Ejercicio 1
    try:
        mostrar_resultados("EJERCICIO 1 - DIAPO 34", EJERCICIO_1, 'S')
    except RecursionError:
        print("\n[!] Error: La Gramatica 1 tiene recursividad izquierda directa.")

    # Ejecutar para Ejercicio 2
    mostrar_resultados("EJERCICIO 2 - DIAPO 35", EJERCICIO_2, 'S')
```

- **Algoritmo FIRST:** Se implementa mediante una recursión con memoria (cache). El algoritmo identifica el conjunto de símbolos terminales que puede iniciar una cadena derivada de un No Terminal, siendo capaz de manejar la propagación del símbolo vacío (épsilon).
- **Algoritmo FOLLOW:** Se implementa mediante un algoritmo de punto de fijación (fixpoint). El programa itera sobre todas las producciones de la gramática actualizando los conjuntos de Siguientes hasta detectar que no hay cambios, permitiendo capturar toda la dependencia de un No Terminal sobre otro.
- **Algoritmo PREDICT:** Este componente toma los resultados de los anteriores. Para cada producción (A -> α), sabemos bajo qué condiciones el parser debe elegir dicha regla, aplicando la unión de (FIRST de α sin épsilon) con el FOLLOW de A en el caso de que la producción sea anulable.

3. El conjunto de datos y las gramáticas:

Si bien la gramática se construye mediante un diccionario de listas, esto permite recorrer las producciones con eficiencia. Se añadieron los casos de prueba de las diapositivas.

- **Prueba 1 (Diapo 34):** Gramática compleja para probar la propagación de épsilon y múltiples terminales.
- **Prueba 2 (Diapo 35):** Gramática que sirve para probar la formación de la tabla de predicción en un contexto de análisis descendente.

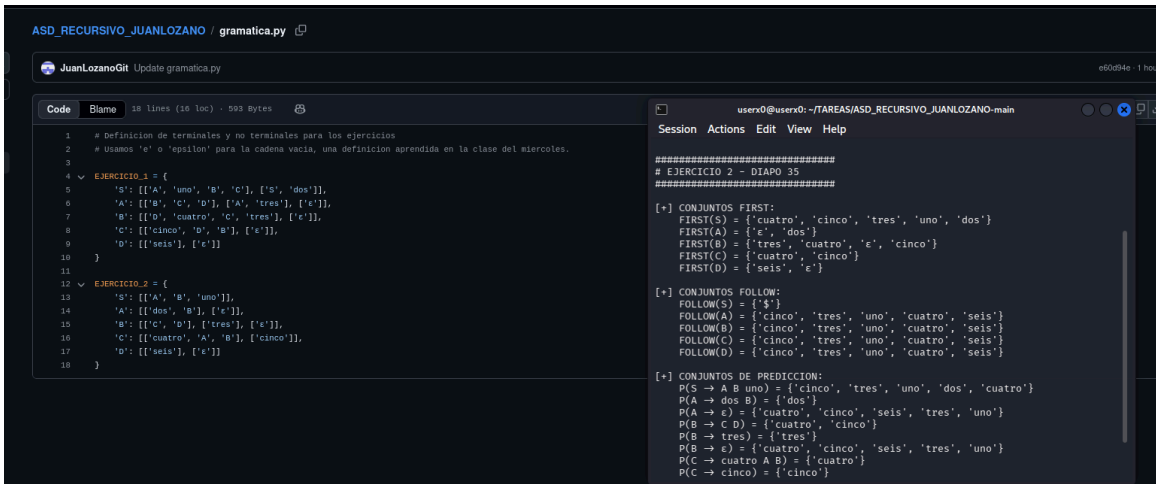
4. Ejecución y modularidad.

El proyecto fue dividido en tres ficheros para asegurar organización:

1. **algoritmos.py:** La lógica matemática y de conjuntos.
2. **gramatica.py:** Las reglas de producción se centralizan (decoupling de datos de la lógica).
3. **main.py:** Actúa como conductor de la ejecución del análisis y lo formatea para su salida en la consola.

5. Análisis de Resultados y Casos Especiales:

Durante las pruebas, se validaron los siguientes puntos críticos:



- **Manejo de Epsilon:** El sistema identifica cuándo un No Terminal es "anulable" y permite que el símbolo inicial de una producción posterior pase al conjunto de Siguientes.
- **Recursividad Izquierda:** Se observó que en la Gramática 1, la regla "S -> S dos" genera una recursividad izquierda directa. El software está diseñado para detectar estas dependencias, haciendo la nota técnica de que para un Parser LL(1) real, estas reglas deberían ser factorizadas.
- **Símbolo de Parada ($):** Se integró automáticamente el marcador "$" en el conjunto FOLLOW del símbolo inicial, cumpliendo con el estándar de los algoritmos de compilación.

6. Conclusion:

1. Gramática del Ejercicio 1: El problema de la Recursividad Izquierda:

- **Hallazgo:** El script detecta cuando intenta calcular los conjuntos de predicción que puede entrar en una especie de ciclo infinito o error de recursión.
- **Causa Técnica:** Esto es a causa de las reglas S → S dos y A → A tres. Este tipo de reglas presentan el problema de que el No Terminal de la izquierda puede aparecer reincidiendo como el primer símbolo de la parte derecha.
- **Resultado:** Un parser descendente trataría de expandir S de modo indefinido, o sea, sin poder consumir ningún token de la producción de la cadena de entrada, lo que haría que no se pueda llegar a construir nunca una tabla de predicción propia de la Gramática LL(1). Para que pueda funcionar, la gramática deberá ser previamente factorizada y/o haber eliminado la recursividad izquierda.

2. Gramática del Ejercicio 2: Gramática LL(1) Válida:

- **Hallazgo:** El script obtenía correctamente y sin ningún problema los conjuntos FIRST, los conjuntos FOLLOW y los conjuntos PREDICT para cada una de las producciones del corpus.
- **Causa Técnica:** Esta gramática, además, no presenta recursividad izquierda y tampoco ambigüedad directa, y los conjuntos FIRST de las distintas producciones para un mismo No Terminal son disjuntos (es decir, no se solapan).
- **Resultado:** Se retornaba que sí, que se trata de una Gramática LL(1), lo que permite construir un parser que, además de ser eficiente, también es determinista.